When you run `bin/rails db:migrate`, the task first looks for the `schema_migrations` table. If it doesn't yet exist, it'll be created.

The migration code then looks at the migration files in `db/migrate` and skips from consideration any that have a version number (the leading digits in the filename) that's already in the database. It then proceeds to apply the remainder of the migrations, creating a row in the `schema_migrations` table for each.

If we were to run migrations again at this point, nothing much would happen. Each of the version numbers of the migration files would match with a row in the database, so there'd be no migrations to apply.

But if we subsequently create a new migration file, it will have a version number not in the database. This is true even if the version number was *before* one or more of the already applied migrations. This can happen when multiple users are using a version control system to store the migration files. If we then run migrations, this new migration file—and only this migration file—will be executed. This may mean that migrations are run out of order, so you might want to take care and ensure that these migrations are independent. Or you might want to revert your database to a previous state and then apply the migrations in order.

You can force the database to a specific version by supplying the `VERSION=` parameter to the `rake db:migrate` command:

```
depot> bin/rails db:migrate VERSION=20250420000009
```

If the version you give is greater than any of the migrations that have yet to be applied, these migrations will be applied.

If, however, the version number on the command line is less than one or more versions listed in the `schema_migrations` table, something different happens. In these circumstances, Rails looks for the migration file whose